Final Report

Siying Liu

1. Introduction

The goal of this project is to implement the Privelet algorithm[1] on dataset adult.data and evaluation. The algorithm adds noise to histogram data while satisfying differential privacy. In this project, I use the Adult dataset and focus on one query: the histogram of education level.

The main idea of differential privacy is to protect each person in the dataset and keep the data as useful as possible. This need us to balance privacy and utility.

In this report, I first explain how I implemented the one-dimensional Privelet algorithm. Then I present the results for different privacy budgets and analyze the quality of the noisy histograms. At the end, I go further by testing more epsilon values.

2. Data and Query H1

For data, I use public dataset adult.data.

The query H1 is the distribution of education level, that is a histogram showing how many records belong to each education category.

I first loaded the dataset in Python and extracted the education column. Then I counted the number of records in each education category. Since the algorithm is designed for one-dimensional ordinal data, I fixed the order of the education levels before building the histogram.

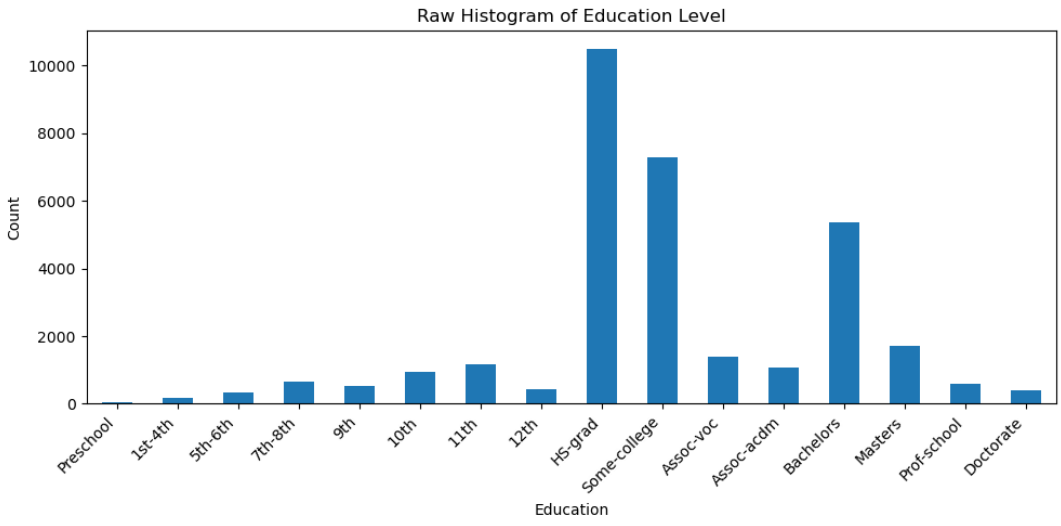

Figure 1. Raw Histogram of Education Level

Figure 1 shows the raw histogram of education level used as the baseline in the following experiments.

3. Implementation of 1D Privelet

To implemented the one-dimensional Privelet algorithm for the histogram of education level, The first step is the wavelet transform. I treated the histogram as the leaves of a full binary tree. Then I grouped values in pairs and computed their averages and differences level by level. The final remaining value is the base coefficient, and all difference values are stored as detail coefficients.

The second step is noise addition. I added Laplace noise to the base coefficient and to the detail coefficients. The noise scale depends on ε, so smaller ε produces stronger perturbation.

The third step is the inverse wavelet transform. Starting from the base coefficient, I reconstructed the histogram from top to bottom by using the detail coefficients. This gives the final noisy histogram.

In code, I implemented these steps with three main functions: “wavelet_transform”, “add_laplace_noise” and “inverse_wavelet_transform”.

4. Results and Evaluation

4.1 Q2: Privelet results for three ε values

For Q2, I used the implemented Privelet algorithm to compute query H1 with three privacy budgets: $\varepsilon = 0.01$, 0.1, and 1.

The results show a clear difference between these three cases**.**

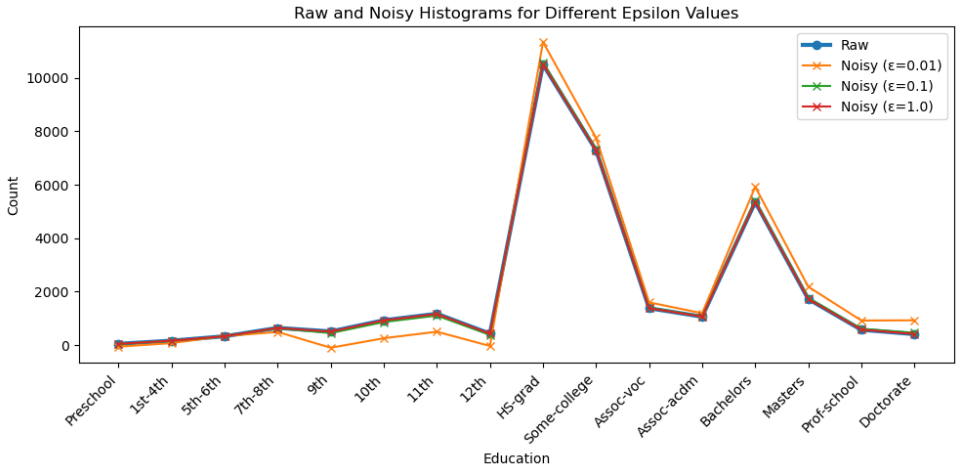

Figure 2. Raw and Noisy Histograms for $\varepsilon = 0.01$, 0.1, and 1

When ε = 0.01, the noise is very strong. The noisy histogram is far from the raw histogram, and some bins even become negative. This means privacy is strong, but the utility is low.

When ε = 0.1, the noisy histogram is closer the raw one, and the result is more stable than ε = 0.01.

When ε = 1, the noisy histogram is much closer to the raw histogram. The global shape of the data is well preserved, so the utility is much better.

These results are expected. A smaller ε adds more perturbation, while a larger ε keeps the result closer to the original data.

4.2 Q3: Experimental evaluation with Wasserstein distance

For Q3, I evaluated the quality of Privelet with the Wasserstein distance between the raw histogram and the noisy histogram. The project requires using ε = 0.01, 0.1, 1, and 10. For each ε value, I ran the algorithm 20 times to make the evaluation more reliable. Then I computed the average, minimum, and maximum Wasserstein distances.

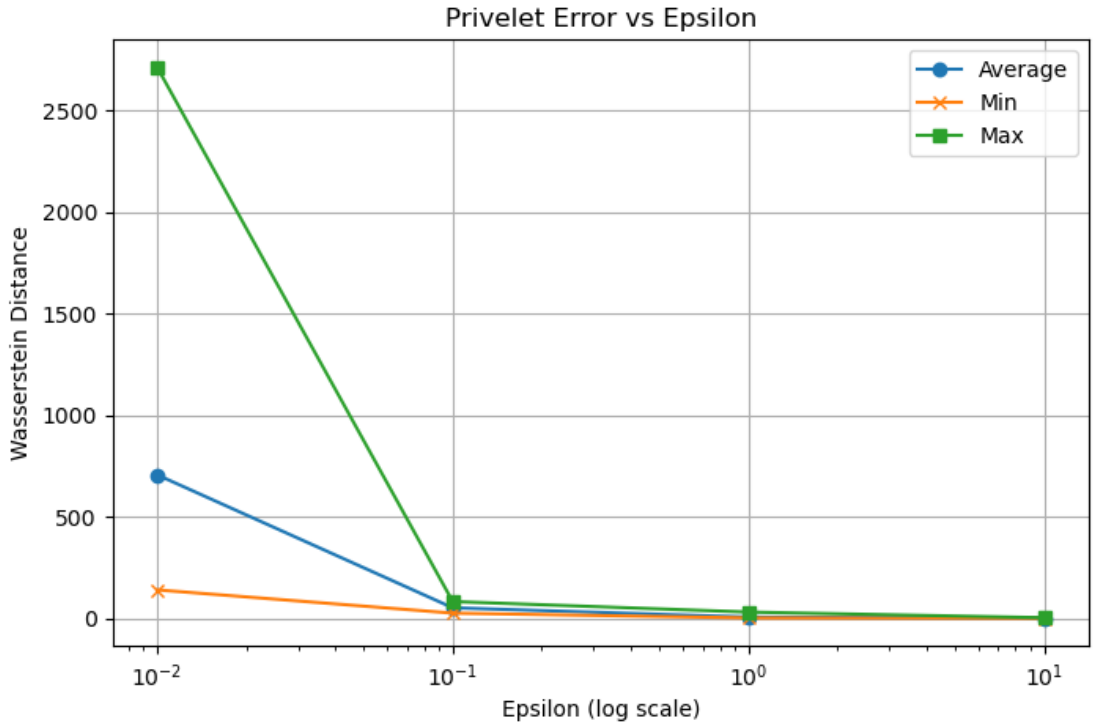

Figure 3. Average, Min, and Max Wasserstein Distance for Different ε

Figure 3 shows a clear privacy-utility trade-off.

From the **whole dataset** point of view, when ε increases, the Wasserstein distance decreases. This means the noisy histogram becomes closer to the raw histogram, so the global

distribution of education levels is better preserved. In other words, the utility improves.

From the **individual data** point of view, when ε is small, more noise is added. This makes it harder to see the effect of any single person in the dataset. Therefore, privacy is stronger. When ε becomes larger, less noise is added, so the final result is more accurate, but the protection of each individual record becomes weaker.

So, the experiment confirms the expected behavior of differential privacy. Small ε means stronger privacy and lower utility, while large ε means weaker privacy and higher utility.

4.3 Further Discussion

To better understand the sharp drop between 10^{-2} and 10^{-1}, I tested more epsilon values in the low range, such as 0.02, 0.03 and so on. Figure 4 shows the results. The extended results show that the Wasserstein distance decreases strongly when epsilon increases from a very small value. In particular, the improvement is very clear between 0.01 and 0.05. After that, the curve becomes smoother.

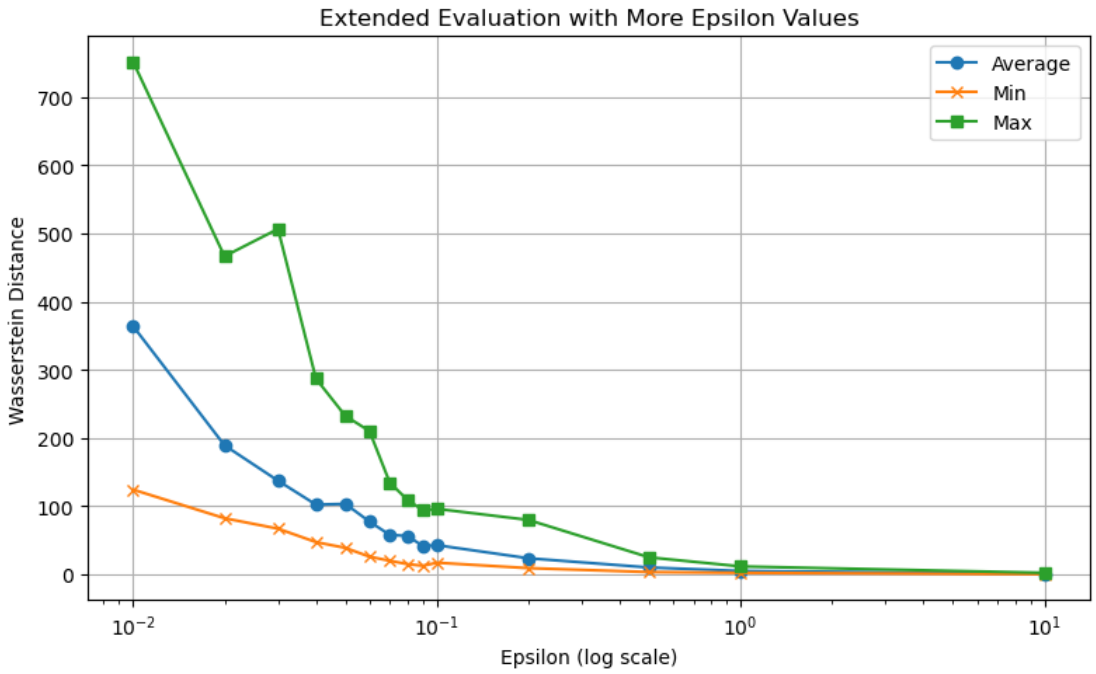

Figure 4. Wasserstein Distance for more ε values

However, the decrease is not perfectly smooth for all measures. The average value shows a clear downward trend, but the minimum and maximum values may fluctuate because the noise is random. This means that the low-epsilon range is more sensitive, but we cannot say that the curve will always have exactly the same sharp shape.

5. Conclusion

In this project, I implemented the one-dimensional Privelet algorithm for ordinal histogram data and applied it to the Adult dataset. The target query was the histogram of education level.

The implementation successfully produced noisy histograms for different privacy budgets. The Q2 results showed that the amount of perturbation strongly depends on ε. The Q3 experiments based on Wasserstein distance and confirmed that utility improves when ε becomes larger. For further research by adding more ε values, we found the Wasserstein distance drop very sharply when ε is in a very small range.

Overall, the project shows that we can protect individual data by adding noise, but stronger privacy usually reduces utility. Privelet offers a structured way to do this for histogram data by using a wavelet-based method instead of adding noise directly to each bin.

References

[1] Xiaokui Xiao, Guozhang Wang, and Johannes Gehrke. “Differential Privacy via Wavelet Transforms”. In: IEEE Trans. Knowl. Data Eng. 23.8 (2011), pp. 1200–1214. DOI: 10.1109/TKDE.2010.247. URL: https://doi. org/10.1109/TKDE.2010.247.